



Esercitazione 12

Wavelets



Introduzione

Questa esercitazione esplora in modo pratico e comparativo le trasformate basate su frequenza e su scale: la **DFT** (Trasformata Discreta di Fourier), la **CWT** (Trasformata Wavelet Continua) e la **DWT** (Trasformata Wavelet Discreta). L'obiettivo è far emergere i pregi e i limiti di ciascun approccio quando si analizzano segnali con componenti localizzate nel tempo (picchi, transitori) e segnali a contenuto variabile di frequenza (es. chirp).

Punti chiave ampliati:

- **Localizzazione tempo vs frequenza:** la DFT dà eccellente risoluzione in frequenza ma *non* localizza gli eventi nel tempo (un impulso appare come spettro esteso con fase dipendente dalla posizione). Le wavelet (CWT/DWT) offrono invece una rappresentazione tempo-scala che evidenzia *quando* avvengono transitori e a quali scale/frequenze sono associati.
- **CWT (mexican-hat / Ricker):** utile per visualizzare picchi singoli o coppie molto ravvicinate; la scelta di scala determina la "risoluzione temporale" (scale piccole → buona risoluzione temporale, scale grandi → buona risoluzione in frequenza). I plotting choices (extent, normalizzazione v_{max} , v_{min}) condizionano la leggibilità delle mappe.
- **DWT (db4, multilevel):** effettua una decomposizione multiscala in coefficienti di approssimazione e dettaglio; è numericamente invertibile — ricostruzione esatta se si mantengono tutti i coefficienti. Questa struttura è ideale per compressione tramite sogliatura dei coefficienti di dettaglio meno importanti.
- **Compressione:** soglia basata sulla media dei valori assoluti dei coeff. di dettaglio (o su metodi più sofisticati come sogliatura universale di Donoho) riduce i coeff. meno significativi e permette una buona ricostruzione con basso MSE; la scelta della soglia e del livello di decomposizione è critica per il tradeoff qualità/ratios di compressione.
- **Conclusione pratica:** usare la DFT per analisi globali di frequenza; usare CWT per studiare transitori e risolvere picchi vicini; usare DWT per compressione, denoising multiscala e ricostruzione efficiente.



Riassunto Esercitazione 12

Obiettivo: confrontare DFT e rappresentazioni wavelet (CWT/DWT) su segnali sintetici con picchi e su un segnale reale (chirp), esplorando decomposizione/ricostruzione e compressione tramite sogliatura dei coefficienti.

Take-home messages generali

- Un impulso nel dominio del tempo corrisponde a uno spettro con magnitude costante (la differenza è nella fase): DFT perde informazione temporale.
- La CWT risolve transitori nel tempo e nello "scale-space": due picchi molto vicini vengono risolti solo a scale fini.
- La DWT fornisce una decomposizione multiscala invertibile: ricostruzione esatta se si mantengono tutti i coefficienti; la compressione è efficace sogliando coefficienti di dettaglio.
- La scelta di wavelet (Haar vs db4 vs altri) e del livello di decomposizione influenza direttamente la qualità della ricostruzione e il rapporto di compressione.
- Per valutare risultati usare sia giudizi visivi (plot) sia metriche quantitative (MSE, eventuale SNR).



Wavelets

1. Principio di incertezza per sequenze finite

- Se un segnale $x \in \mathbb{C}^N$ ha N_t valori non nulli e la sua DFT X ha N_f valori non nulli, allora:

$$N_t N_f \geq N$$

- Dimostrazione: impossibile avere troppi zeri consecutivi nella DFT se il segnale è sparso.
- L'uguaglianza si può raggiungere in casi specifici.

2. Confronto tra DFT e DWT

- **DFT:** rappresenta il segnale come combinazione di funzioni sinusoidali (analisi in frequenza). È invertibile.
- **DWT:** rappresenta il segnale come combinazione di *wavelet* localizzate (analisi tempo-frequenza). È invertibile.
- Le wavelet sono versioni dilatate, traslate e scalate di una *mother wavelet*.
- Esempi di famiglie di wavelet: Haar, Daubechies.

3. DWT monodimensionale

- Funziona su segnali di lunghezza 2^n .
- Divide il segnale in:
 - **coeff. di approssimazione** (low-pass, basse frequenze).
 - **coeff. di dettaglio** (high-pass, alte frequenze).
- Output = stessa lunghezza del segnale originale.

4. Trasformata a più livelli

- Si applicano successivamente più filtri:
 - L_3 : tre filtri passa-basso consecutivi,
 - H_3 : due passa-basso + uno passa-alto,

- H_2 : un passa-basso + uno passa-alto,
- H_1 : un passa-alto.
- Isolamento delle frequenze:
 - H_1 : alte frequenze,
 - H_2 : frequenze intermedie,
 - L_3 : basse frequenze.

5. Coefficienti di filtro

- Dalla coppia (filtro passa-basso h , filtro passa-alto g) derivano basi ortogonali e compatte.
- Relazione:

$$g_k = (-1)^k h_{n-k-1}$$

- Esempi:
 - Haar ($n = 2$): $h = [1/\sqrt{2}, 1/\sqrt{2}]$,
 - Daubechies-4 ($n = 4$): coefficienti più complessi.

6. Inverse DWT

- Permette di ricostruire il segnale originale senza perdita di informazione.

7. DWT bidimensionale (immagini)

- Si applica la DWT prima sulle **righe**, poi sulle **colonne**.
- Risultano 4 bande:
 - **LL**: versione ridotta dell'immagine,
 - **LH**: evidenza dettagli orizzontali,
 - **HL**: evidenza dettagli verticali,
 - **HH**: evidenza dettagli ad alta frequenza (punti/localizzazioni).

```
In [2]: import numpy as np
import matplotlib.pyplot as plt
from scipy import fftpack as f
import scipy.signal as ssi
import scipy.io as sio
import pywt
import scipy.io.wavfile
```

🔴 Esercizio 1. DFT e DWT

(a) Considerare un segnale di dimensione 512 con tutti zeri ed un picco al campione 200 che vale 50. Effettuare la FFT del segnale e visualizzare il modulo dei coefficienti ottenuti. Ripetere lo stesso esercizio con un segnale con un picco a 100.

(b) Il comando `cwt(x, scipy.signal.ricker, np.arange(1,48))` della libreria `scipy.signal` calcola la trasformata wavelet continua del segnale `x` (wavelet "ricker" = Mexican hat). Utilizzare `cwt` per calcolare la trasformata wavelet continua dei due segnali introdotti precedentemente e fare un plot dei coefficienti ottenuti.

(c) Costruire un segnale con due picchi molto vicini e calcolare la trasformata wavelet continua, visualizzare i coefficienti ottenuti e osservare come cambia il plot all'avvicinarsi dei due picchi.

```
In [ ]: # Esercizio 1(a) - Due segnali con picchi singoli

# Segnale 1: picco a 200
x1 = np.zeros(512)
x1[200] = 50
X1 = f.fft(x1)

# Segnale 2: picco a 100
x2 = np.zeros(512)
x2[100] = 50
X2 = f.fft(x2)

# Plot nel dominio del tempo
plt.figure()
plt.plot(x1, label='x1 (picco a 200)')
plt.plot(x2, label='x2 (picco a 100)')
plt.xlabel('Indice temporale')
plt.ylabel('Ampiezza')
plt.title('Segnali nel dominio del tempo')
plt.legend()
plt.grid(True)
plt.show()

# Plot nel dominio delle frequenze
freqs = np.arange(512) / 512 # Frequenze normalizzate
plt.figure()
plt.plot(freqs, np.abs(f.fftshift(X1)), label='|X1|')
plt.xlabel('Frequenze normalizzate')
plt.ylabel('Ampiezza')
plt.title('DFT di x1')
```

```

plt.grid(True)
plt.show()

plt.figure()
plt.plot(freqs, np.abs(f.fftshift(X2)), label='|X2|', color='orange')
plt.xlabel('Frequenze normalizzate')
plt.ylabel('Ampiezza')
plt.title('DFT di x2')
plt.grid(True)
plt.show()

# Esercizio 1(b) - Trasformata wavelet continua

# Wavelet Mexican hat ("mexh" in PyWavelets), scale da 1 a 47
scales = np.arange(1, 48)

coeffs_X1, freqs_X1 = pywt.cwt(x1, scales, 'mexh')
plt.imshow(coeffs_X1, extent=[0, 512, scales[-1], scales[0]],
           aspect='auto', cmap='jet',
           vmax=np.abs(coeffs_X1).max(), vmin=-np.abs(coeffs_X1).max())
plt.colorbar(label='Coefficiente wavelet')
plt.title('CWT - x1')
plt.xlabel('Indice temporale')
plt.ylabel('Scala')
plt.show()

coeffs_X2, freqs_X2 = pywt.cwt(x2, scales, 'mexh')
plt.imshow(coeffs_X2, extent=[0, 512, scales[-1], scales[0]],
           aspect='auto', cmap='jet',
           vmax=np.abs(coeffs_X2).max(), vmin=-np.abs(coeffs_X2).max())
plt.colorbar(label='Coefficiente wavelet')
plt.title('CWT - x2')
plt.xlabel('Indice temporale')
plt.ylabel('Scala')
plt.show()

# Esercizio 1(c) - Due picchi vicini

def plot_wavelet_signal(sig, title):
    coeffs, freqs = pywt.cwt(sig, scales, 'mexh')
    plt.imshow(coeffs, extent=[0, 512, scales[-1], scales[0]],
               aspect='auto', cmap='jet',
               vmax=np.abs(coeffs).max(), vmin=-np.abs(coeffs).max())
    plt.colorbar(label='Coefficiente wavelet')
    plt.title(title)
    plt.xlabel('Indice temporale')
    plt.ylabel('Scala')
    plt.show()

# Caso 1: picchi a 100 e 200
x3 = np.zeros(512)
x3[100] = 50
x3[200] = 50
plot_wavelet_signal(x3, 'CWT - Picchi a 100 e 200')

# Caso 2: picchi a 100 e 80
x3 = np.zeros(512)
x3[100] = 50
x3[80] = 50
plot_wavelet_signal(x3, 'CWT - Picchi a 100 e 80')

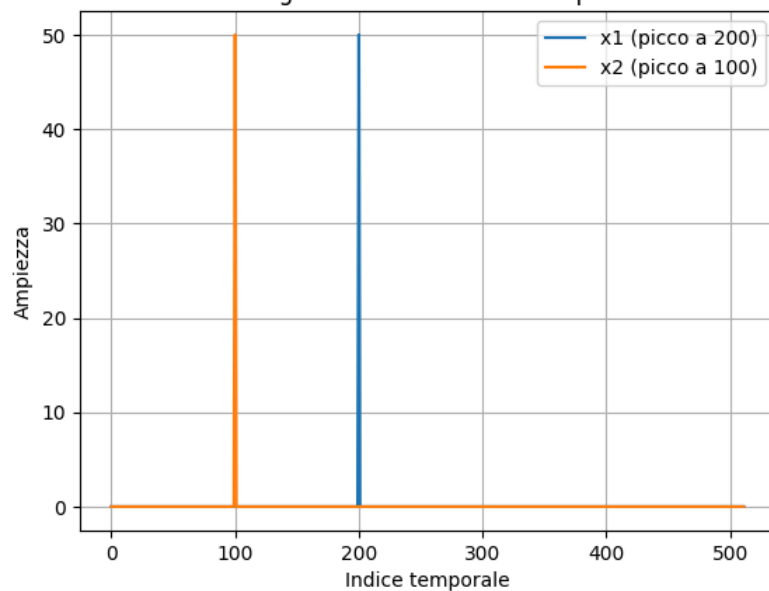
# Caso 3: picchi a 100 e 90
x3 = np.zeros(512)
x3[100] = 50
x3[90] = 50
plot_wavelet_signal(x3, 'CWT - Picchi a 100 e 90')

# Caso 4: picchi a 100 e 95
x3 = np.zeros(512)
x3[100] = 50
x3[95] = 50
plot_wavelet_signal(x3, 'CWT - Picchi a 100 e 95')

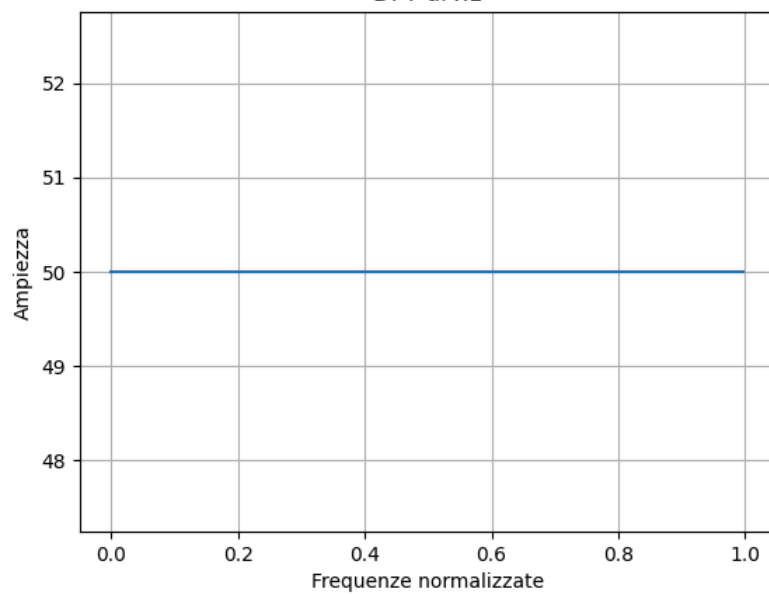
# Caso 5: picchi a 100 e 99
x3 = np.zeros(512)
x3[100] = 50
x3[99] = 50
plot_wavelet_signal(x3, 'CWT - Picchi a 100 e 99')

```

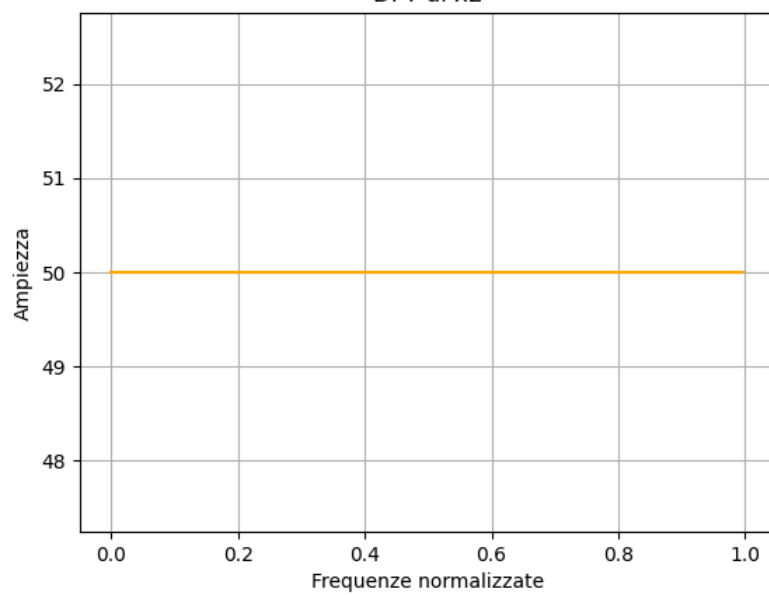
Segnali nel dominio del tempo

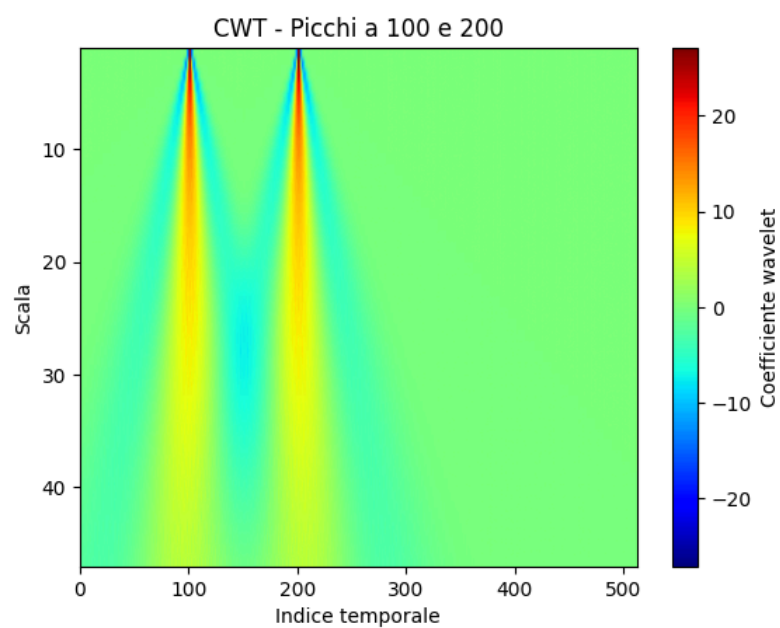
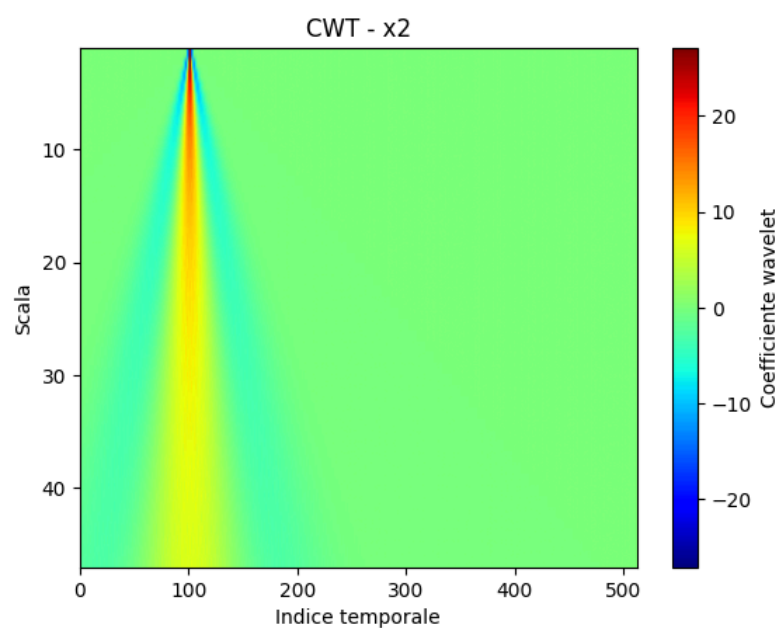
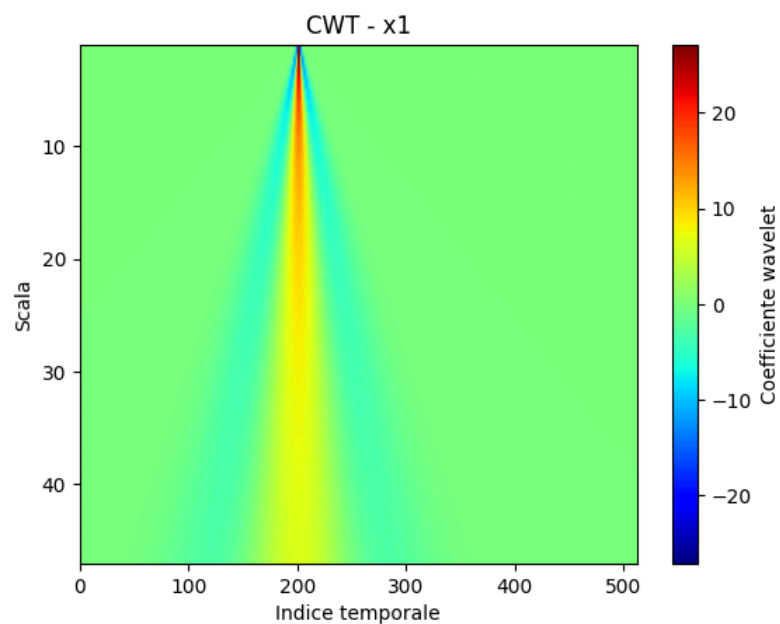


DFT di x1

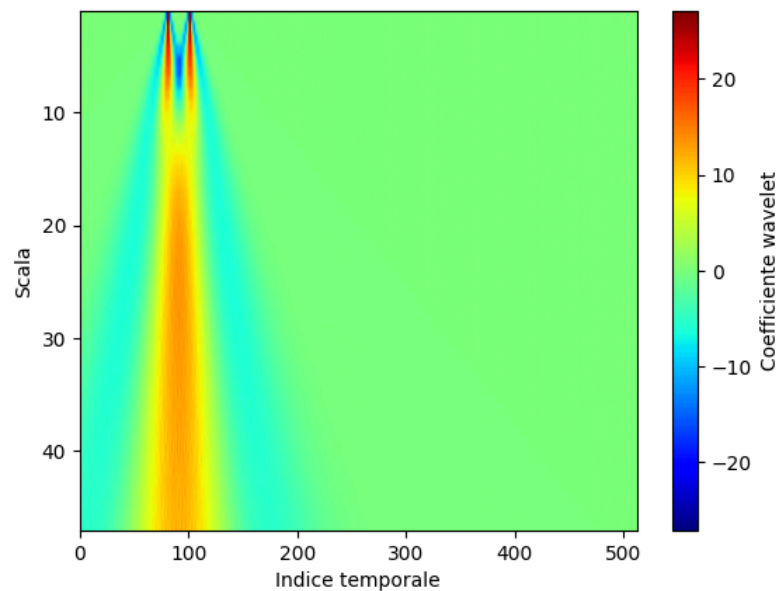


DFT di x2

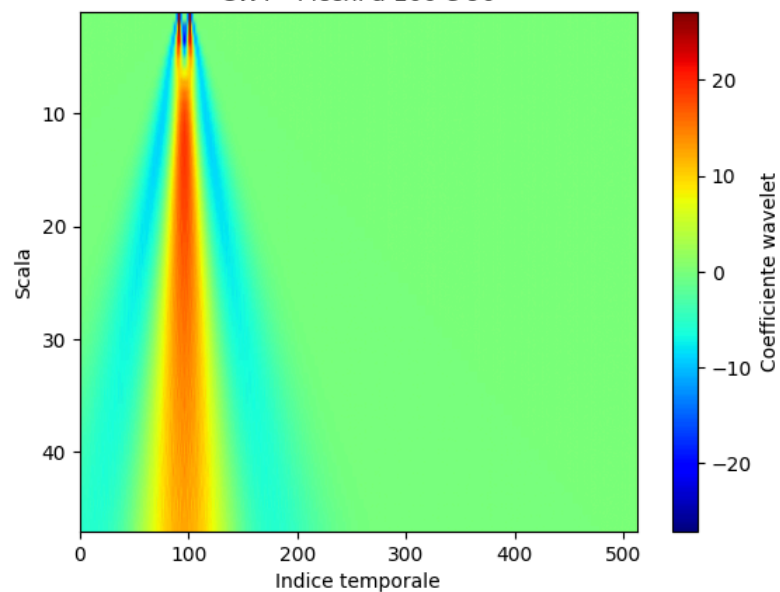




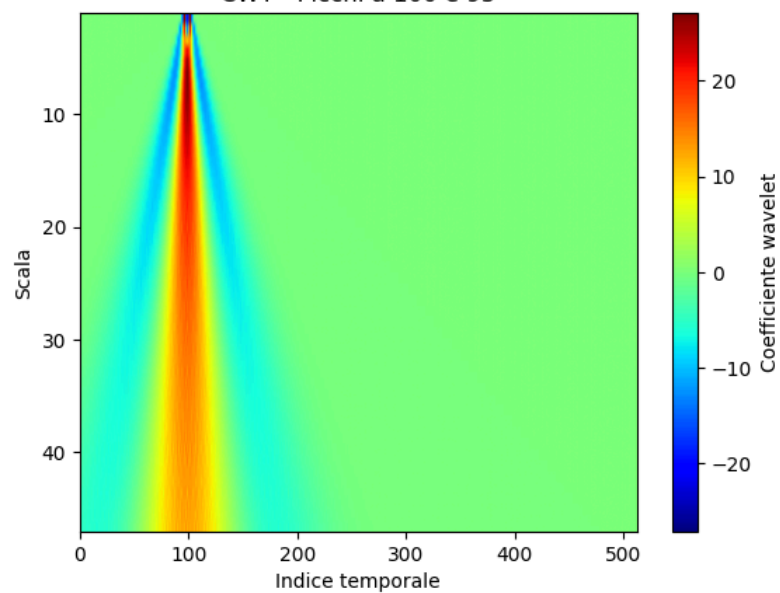
CWT - Picchi a 100 e 80



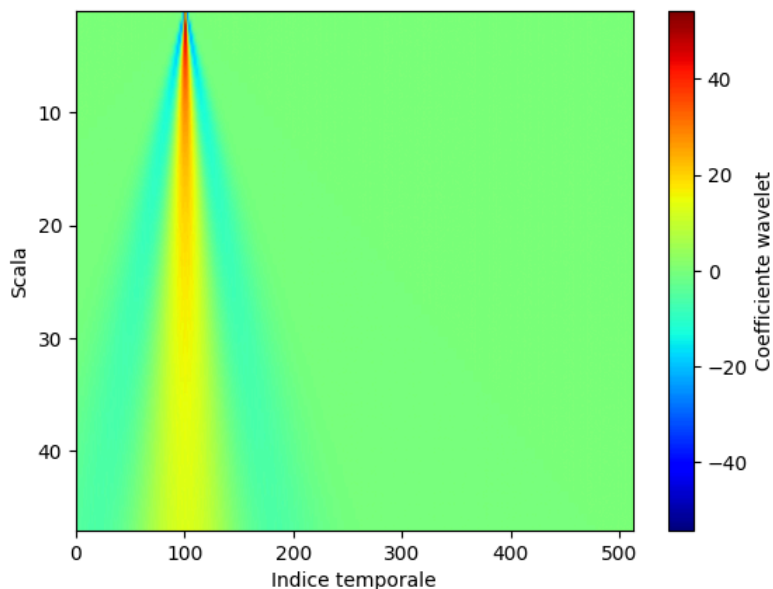
CWT - Picchi a 100 e 90



CWT - Picchi a 100 e 95



CWT - Picchi a 100 e 99



COMMENTO.

Obiettivi

Studiare come un impulso e coppie di impulsi vengono rappresentati nel dominio delle frequenze (DFT) e nel dominio tempo-scala (CWT). Confrontare la capacità delle due trasformate di fornire informazioni su **contenuto in frequenza e localizzazione temporale**.

Commento sui plot

- **Grafico: FFT di un impulso a 100 e a 200** Le due FFT mostrano magnitudine piatta: la posizione temporale del picco non si riflette nello spettro di ampiezza, ma solo nella fase. La DFT conferma che un impulso ha energia distribuita su tutte le frequenze, ma non dice *dove* è avvenuto.
- **Grafico: CWT di un impulso singolo** La mappa mostra una risposta netta centrata nell'indice del picco. L'intensità si concentra a scale piccole (ottima risoluzione temporale) e si allarga a scale maggiori. Qui emerge la capacità delle wavelet di localizzare eventi nel tempo.
- **Grafico: CWT di due impulsi distanti (100 e 200)** Si vedono due creste distinte nelle mappe tempo-scala. Gli impulsi sono risolti chiaramente.
- **Grafico: CWT di due impulsi più vicini (100 e 90)** Le due risposte si sovrappongono parzialmente: a scale grosse i due impulsi si fondono, a scale fini rimangono distinguibili.
- **Grafico: CWT di due impulsi quasi coincidenti (100 e 99)** Qui la risoluzione è al limite: i due picchi sono separabili solo a scale molto fini, mentre già a medie scale diventano praticamente indistinguibili.

Take-home message

La DFT fornisce una rappresentazione globale in frequenza senza informazione temporale. La CWT, invece, permette di osservare *quando* accadono gli impulsi e di distinguere eventi ravvicinati a scale sufficientemente piccole. La risoluzione temporale e quella in frequenza sono un compromesso governato dalla scala scelta.

✦ Esercizio 2. DWT e compressione

(a) Leggere il file `chirp.mat` utilizzando il comando:

```
np.squeeze(sio.loadmat('chirp.mat')['x'])
```

Fare un plot del segnale contenuto nel file.

(b) Con il comando `dwt(x, 'db4')` della libreria `pywt` ottenere la scomposizione al primo livello del segnale; il comando restituisce due vettori contenenti i coefficienti dell'approssimazione e del dettaglio. Utilizzando `upcoef` ottenere le approssimazioni `a` e i dettagli `d` al primo livello.

(c) Utilizzando `idwt` ricostruire il segnale. Confrontare il risultato ottenuto con la somma `d + a`, dove `d` e `a` sono i dettagli e le approssimazioni calcolate al punto precedente.

(d) Utilizzare `wavedec` della libreria `pywt` per avere più livelli di decomposizione (provare con 3 livelli).

(e) Comprimer il segnale eliminando i coefficienti di dettaglio meno espressi: calcolare la media del valore assoluto dei coefficienti di dettaglio al terzo livello ed azzerare i coefficienti inferiori a tale valore. Ricostruire il segnale con `waverec` e calcolare l'errore quadratico medio (MSE) rispetto al segnale originale.

```
In [ ]: # Funzione per salvare l'audio in formato int16
def save_wav(filename, data, fs):
    data_norm = data / np.max(np.abs(data)) # normalizza a [-1, 1]
    data_int16 = np.int16(data_norm * 32767)
    scipy.io.wavfile.write(filename, fs, data_int16)

# Esercizio 2(a) - Lettura e visualizzazione del segnale
x = np.squeeze(sio.loadmat('chirp.mat')['x'])
save_wav('chirp.wav', x, 9000)

plt.figure()
plt.plot(x)
plt.title('Segnale originale (chirp)')
plt.xlabel('Campioni')
```

```

plt.ylabel('Ampiezza')
plt.grid(True)
plt.show()

# Esercizio 2(b) - DWT e ricostruzione delle componenti
cA, cD = pywt.dwt(x, 'db4')

A = pywt.upcoef('a', cA, 'db4', take=len(x))
D = pywt.upcoef('d', cD, 'db4', take=len(x))

plt.figure()
plt.plot(A, label='Approssimazione (A)')
plt.plot(D, label='Dettaglio (D)')
plt.title('Componenti al primo livello')
plt.xlabel('Campioni')
plt.ylabel('Ampiezza')
plt.legend()
plt.grid(True)
plt.show()

# Esercizio 2(c) - Ricostruzione e confronto
x_rec = pywt.idwt(cA, cD, 'db4')

err = np.abs(A + D - x_rec)
print(f"Max error: {np.max(err):.6e}")

plt.figure()
plt.plot(err)
plt.title('Errore tra (A+D) e IDWT')
plt.xlabel('Campioni')
plt.ylabel('Errore')
plt.grid(True)
plt.show()

# Esercizio 2(d) - Decomposizione multilevel
cA, cD3, cD2, cD1 = pywt.wavedec(x, 'db4', level=3)

# Esercizio 2(e) - Compressione basata sulla soglia
threshold = np.mean(np.abs(cD3))

def compress_coeffs(cD, thr):
    cD_c = np.zeros_like(cD)
    idx = np.where(np.abs(cD) > thr)
    cD_c[idx] = cD[idx]
    return cD_c

cD1_c = compress_coeffs(cD1, threshold)
cD2_c = compress_coeffs(cD2, threshold)
cD3_c = compress_coeffs(cD3, threshold)

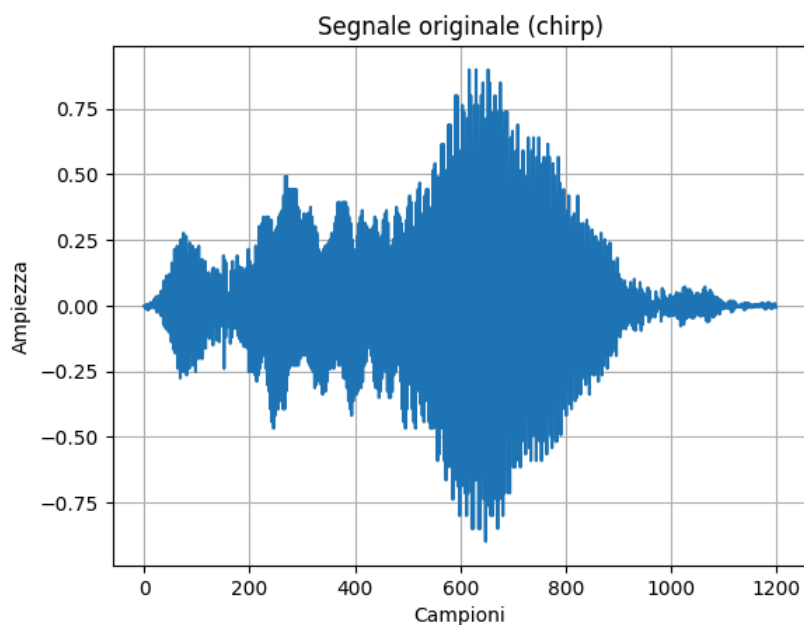
x_rec2 = pywt.waverec((cA, cD3_c, cD2_c, cD1_c), 'db4')

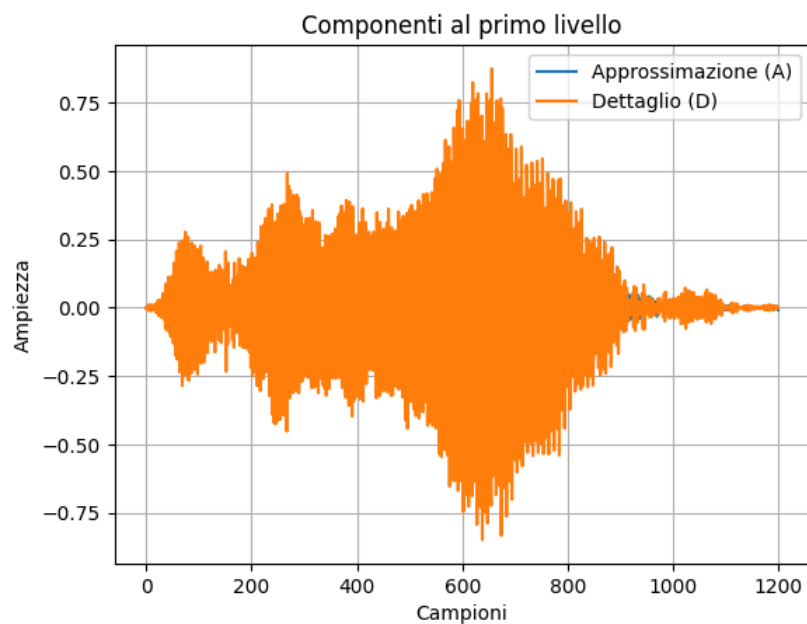
save_wav('chirp_compresso.wav', x_rec2, 9000)

mse_error = np.mean((x - x_rec2) ** 2)
print(f"Errore di approssimazione (MSE) = {mse_error:.6e}")

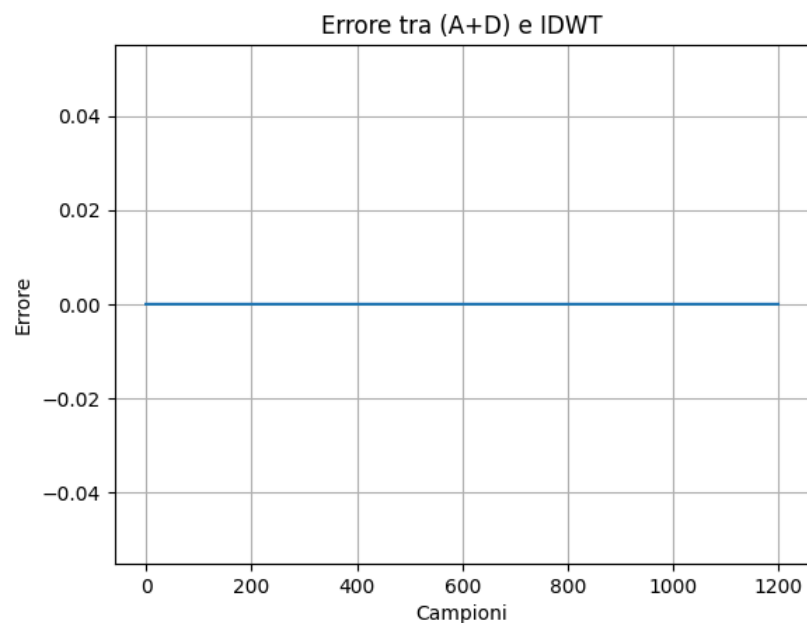
plt.figure()
plt.plot(x, label='Originale')
plt.plot(x_rec2, label='Compresso', alpha=0.7)
plt.title('Segnale originale vs compresso')
plt.xlabel('Campioni')
plt.ylabel('Ampiezza')
plt.legend()
plt.grid(True)
plt.show()

```

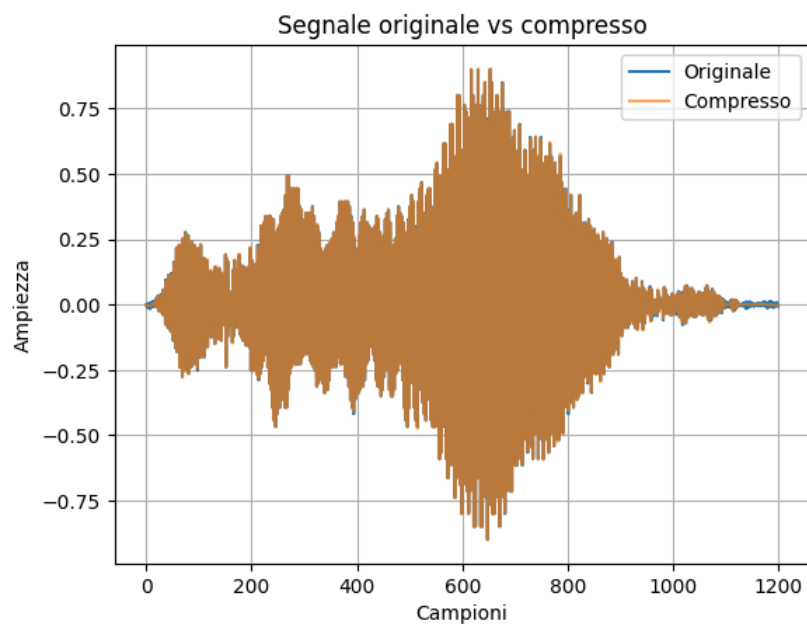




Max error: 0.000000e+00



Errore di approssimazione (MSE) = 1.465604e-05



COMMENTO.

Obiettivi

Applicare la DWT al segnale chirp per studiarne la scomposizione in approssimazioni e dettagli, verificare la ricostruzione esatta a partire dai coefficienti, realizzare una decomposizione multiscala e sperimentare una compressione basata sulla soglia di attivazione dei coefficienti.

Commento sui plot

- **Grafico: Segnale chirp** Si osserva la frequenza istantanea crescente (oscillazioni che diventano via via più fitte). È un buon segnale test per verificare la capacità di un metodo di analizzare contenuto variabile nel tempo.
- **Grafici: Approssimazione (A) e Dettaglio (D) al primo livello** L'approssimazione cattura le variazioni lente (bassa frequenza), mentre il dettaglio isola le oscillazioni rapide. La somma A+D segue da vicino l'andamento del segnale originale.
- **Grafico: Ricostruzione e confronto con idwt** La differenza è praticamente zero (errore massimo riportato $\sim 10^{-16}$): la DWT/IDWT è invertibile e mantiene perfettamente l'informazione.
- **Grafici: Decomposizione multilevel (livelli 3)** I dettagli ai livelli 1, 2, 3 mostrano contenuti a bande diverse, con oscillazioni sempre più lente man mano che il livello aumenta. L'approssimazione al livello 3 è molto smussata: rimangono solo le componenti a bassissima frequenza.
- **Grafico: Segnale originale vs segnale compresso** Dopo la soglia di attivazione dei coefficienti di dettaglio, i due segnali risultano quasi sovrapposti. L'MSE riportato ($\sim 1.46 \times 10^{-5}$) conferma che la perdita è minima.

Take-home message

La DWT consente di separare un segnale in componenti a diversa scala e ricostruirlo senza perdita. La compressione tramite soglia di attivazione riduce i coefficienti mantenendo un'ottima fedeltà (basso MSE). È quindi uno strumento potente non solo per l'analisi multiscala ma anche per applicazioni pratiche come compressione e denoising.